

ASSEMBLY SYSTEM ANALYSIS AND DESIGN: WEBOTS PATH FINDING WITH A* ALGORITHM

ABSTRACT

The project saw the implementation of a MATLAB controller on a designed robotic vehicle in Webots. Obstacle avoidance was conducted utilising dynamic mapping of the environment using distance sensors on the vehicle and an A* algorithm was implemented to conduct path planning. The system was tested in three different environments and successful navigation of the vehicle to the goal was recorded in each. At each goal, a grabber system placed a box on the vehicle successfully.

TABLE OF CONTENTS

Abstract	1
Introduction.....	1
Vehicle Design	1
Implementation – “vehicle_controller.m”	3
System Overview	3
Initialisation	3
Grid Mapping.....	3
A* Algorithm Implementation – “aStarSearch”	4
Core Loop and Grabber Functionality	5
Testing and Conclusion	6
References	6

INTRODUCTION

Webots allows for the utilisation of modelling to simulate robotic systems (Cyberbotics, 2026). One such implementation of simulation is path finding, notably the A* Algorithm. The A* Algorithm was first published by Peter Hart, Nils Nilsson and Bertram Raphael as a way of “determining the minimum cost path through a graph” using a heuristic (Hart, et al., 1968). It works by using a heuristic to estimate the distance from a start position to the goal as well as the cost incurred by reaching the starting position. As such, it can estimate the fastest path to the goal without exploring all nodes in the graph increasing performance. It was implemented in Webots using a MATLAB Controller to control and determine the optimal path for a 4 wheel robotic vehicle.

VEHICLE DESIGN

The design of the vehicle is inherent to the success of the pathfinding. The robotic system will not be given knowledge of the environment prior to runtime, as such it requires obstacle detection functionality perform avoidance functions. It also must remain stable, as well as provide forward momentum and turning functionality. Thus, a system comprising of a main body and four individually controlled motors with attached wheels. Affixed onto the front of the body are three distance sensors with a max range of 0.3m. Figure 1 below illustrates the design of the robotic system as well as a drawing of its technical specifications using Autodesk Fusion:

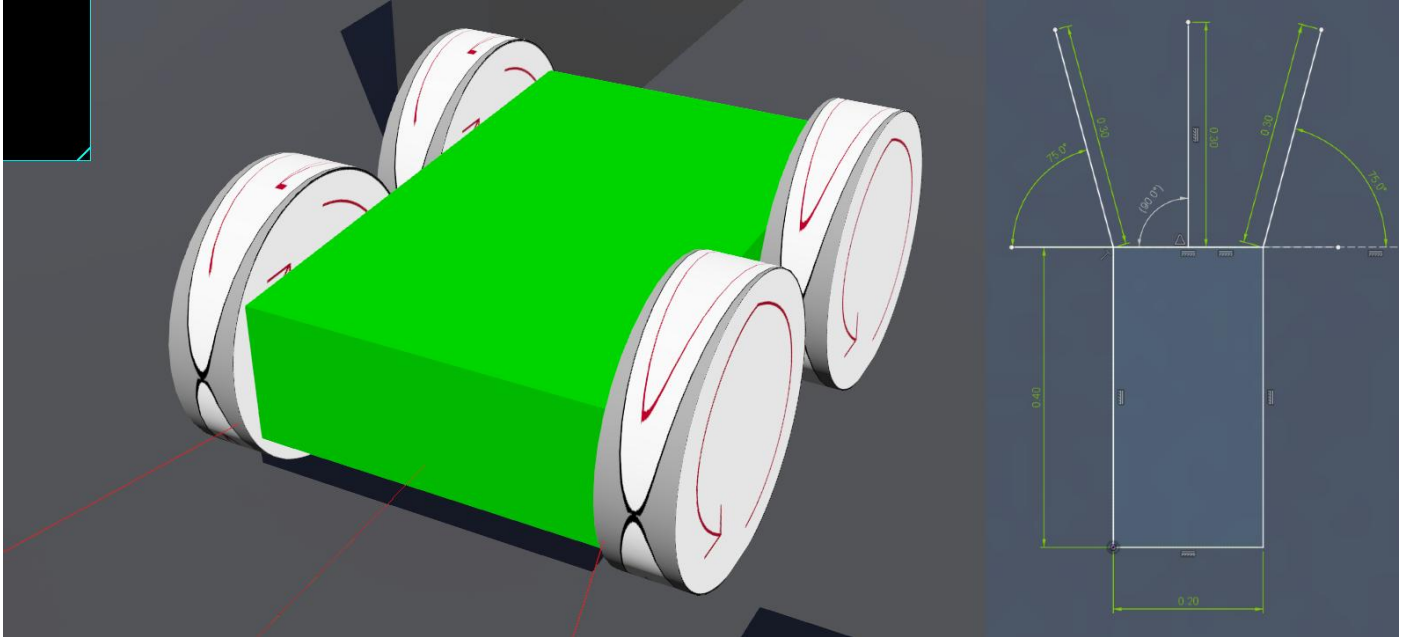


Figure 1 - Design of the Robotic System and its Technical Specifications

The drawing does not feature the robots height (0.1m) as well as the specifications of its wheels which are modelled as rotated cylinders with a diameter of 0.2m and height of 0.05m. In Webots, the main body consists of a solid box (0.4m x 0.2m x 0.1m). Attached at the ends of both long edges are the four wheels which are attached using hinge joints allowing for differential drive capability. As aforementioned, the robot has three distance sensors placed at the front of the robot. The left and right distance sensors are rotated slightly (75 degrees) to allow for mitigation of obstacle collisions of the wheels. A GPS sensor is placed at its central position, elevated to the top of the main chassis (0,0,0.05) as well as an Inertial sensor to provide orientation information. An emitter and receiver are also implemented to provide communication with other robotic devices (such as the grabber later mentioned).

The design of the robot meets the technical requirements of the coursework specified. The distance sensors allow for obstacle detection however a lidar sensor may be more appropriate, allowing for an increased range of detection without the implementation of excessive distance sensors. Furthermore, the wheel and chassis size could be reduced to facilitate a successful testing of an A* path finding robotic system in a much smaller environment.

IMPLEMENTATION – “VEHICLE_CONTROLLER.M”

SYSTEM OVERVIEW

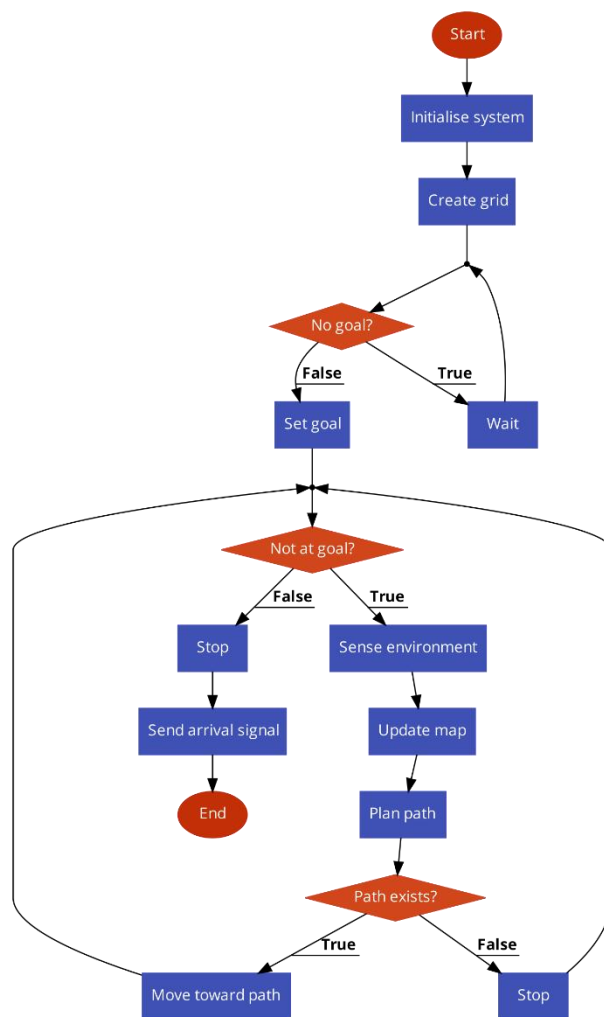


Figure 2 - Flowchart Representation of Overall System

A flowchart illustration of the system is depicted in Figure 2. The system is initialised before creating a smaller grid using the dimensions of the simulation environment for later path finding utilisation. The system awaits and obtains a goal position from the grabber which the robot then navigates towards. It utilises environmental mapping using the distance sensors and calculates a recommended path using the A* algorithm with Euclidean Distance as a heuristic. Once arriving at the grabber system, it emits a corresponding message and halts the wheels, ceasing movement. The grabber system then picks and places a target box onto the robotic vehicle.

INITIALISATION

The implementation of a path finding robotic system with obstacle avoidance was achievable using a single MATLAB controller. A simulation timestep of 64ms was incorporated to allow for efficiency during runtime as well as providing a large enough sample rate to allow for accurate testing. All wheels (front-left, front-right, back-right, back-left) were obtained, as well as the distance sensors, GPS sensor, inertial unit sensor, emitter and receiver. All sensors and motors were enabled using the corresponding MATLAB Webots function (e.g. “wb_distance_sensor_enable”) alongside the timestep. An exception exists for the rotational motors of the wheels which were set to velocity control (“initialiseWheels” function) mode allowing for the wheels to be controlled by speed rather than a set target angle, which is more appropriate for this use case.

GRID MAPPING

The utilisation of the A* algorithm requires a node map specifying the location of free space, starting position, end goal and obstacles. This is implemented as a two-dimensional, top-down view grid which is an abstraction of the operable space in which the robot is situated in. The user must input parameters for the dimensions of the arena (“floor_x” and “floor_y” as well as a suitable “cell_size” which relates to the size of each cell of the grid. For example, a cell size of 0.3 ensures that each cell of the grid represents a 0.3m x 0.3m subset of the arena. Subsequently, a message is awaited and received from the gripper, containing its x and y position. This is then converted into a coordinate based on the grid (e.g. “floor((goal_x + floor_x/2) / cell_size) + 1;”) by ensuring the value of the position is not below 1 (MATLAB array specifications) due to the arena being centred at (0,0,0) and appropriately scaled using the cell size ensuring it correctly corresponds to its grid representation. In fact, this process is repeated throughout the code when converting real positions in the simulation to the abstracted grid representation. Elements on the grid are given a key which is held as an integer value in the grid. “0” = free space, “1” = obstacle, “2” = goal and “3” = near obstacle.

As the robotic vehicle is not provided with knowledge about the environment (aside from its dimensions) it relies on obstacle detection to map the environment. The main functionality is implemented in the “mapGridCoordinates” function. The function is given the robots position, orientation and distance sensors as inputs, as well as the arenas dimensions, the cell size of the grid, the grid itself and an obstacle enlarging amount. Utilising the technical specifications of the robot (as illustrated in Figure 1), we can ascertain the relative positions of the end of the distance sensors from the GPS sensor and provide them each with a mathematical model. It is incumbent upon the sensor reading of the distance sensor (0-1000) which relates to the position of detection (scaled along 0.3 metres) and the angle at which the sensor is

oriented away from the GPS sensor. The relative positions of the distance sensors from the GPS sensor as an origin is given below in Table 1:

Distance Sensor	Translation Relative to GPS Sensor (m)	Angle Relative to GPS Sensor (degrees)
Middle	(0,0.2)	0 degrees (0 rad)
Left	(-0.125,0.2)	105 degrees (1.83 rad)
Right	(0.125,0.2)	75 degrees (1.31 rad)

Table 1 - Relative Positions of the Distance Sensors for use with Grid Mapping

Using trigonometric measurements, including the reading of the distance sensor (e.g. 500 = 0.15m), encountered obstacle positions can be correctly identified.

The robotic vehicle itself will likely be situated at different positions, in different orientations and thus the positions of the obstacles need to be corrected for its situation on the grid. Using the input orientation obtained from the inertial sensor, a two dimensional rotation of the position of the detected obstacle is implemented to derive its real world position. Now that the position is correctly oriented, it needs to account for the robotic vehicles position in the real world resulting in offsetting by the GPS sensors coordinates. Now, the x and y position of the encountered obstacle is correctly identified. To translate this real world position into its grid position, the process mentioned earlier is implemented. This process is repeated for all x/y coordinates of each distance sensors and allows for at most, three detections to be achieved in any given simulation step.

If the grid position of the detected obstacle is not the goal position, the “addObstacle” function is enacted with the obstacles position, the grid itself and the value of the obstacle enlarging amount. The grid is updated with the obstacles position being denoted as “1” with a square using the position as a centre, and its defined edges by the input obstacle enlarging amount, with values of “3” denoting an area around the obstacle. This is the key user defined variable to adjust obstacle avoidance. The later described A* algorithm implementation utilises this value to adjust course, mitigating contact with obstacles. Larger values will increase distance from obstacles however path finding may prove to take inefficient pathways.

An optional debug display was included, as a Display node on the robotic vehicle, which takes the input grid and displays it for the user (“updateGridDisplay” function). The display indicates the A* algorithm path (red), the robotic vehicle location (green), the goal location (yellow) and any detected obstacles (black). An example of its output is illustrated in Figure 3:

This sequence of grid mapping is performed on every simulation step allowing for accurate mapping of the environment.

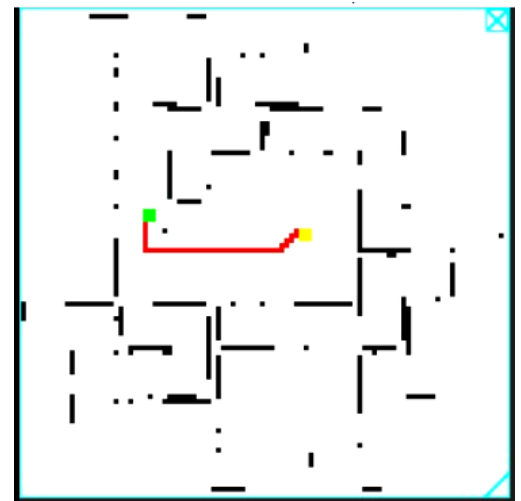


Figure 3 - Live Display of Grid during Runtime

A* ALGORITHM IMPLEMENTATION – “ASTARSEARCH”

The implementation of the A* algorithm is highly dependent on the functionality of the input grid and is largely contained within the “aStarSearch” function. The robotic vehicles position and the goal position (both converted to grid coordinates) are taken as inputs as well. The function initialises a series of arrays containing different information. Table 2 below describes each array:

Array	Description
“unexplored”	False initiated array which tracks cells that need to be checked
“unexplored_list”	Separate array containing unexplored cells, created for efficiency purposes (prevents searching entire grid for next best cell)
“explored”	False initiated array which tracks cells that have been already checked

"f_costs"	Stores estimated cost of each cell
"g_costs"	Stores cost from start to cell
"parent_x" and "parent_y"	Stores previous cell information, utilised to generate path

Table 2 - A* Algorithm Implementation Arrays Description

The algorithm begins with the start cell (the position of the robot). The "unexplored" array has the start cell's x/y value set to True, the "g_cost" arrays x/y value set to 0 and the "f_cost" arrays value set to the value given by a separate "getHeuristic" function which returns the Euclidean distance of the cell ("heuristic = $\sqrt{(x1 - x2)^2 + (y1 - y2)^2}$;" after being given the cell position and goal position and the "unexplored_list" array is given the position of the start cell. The algorithm subsequently performs a while loop over the "unexplored_list" array until it is empty, which is not the case until all relevant cells have been explored. In the list, it chooses the cell with the lowest f_cost value and, if it is not the goal cell (in which case the loop breaks), removes it from the unexplored list, into the explored list. A separate "getNeighbours" function returns the neighbouring cells around the cell in question, in the cardinal directions if it can from the grid. For each neighbouring cells, a series of operations take place. If the neighbour has already been explored, it is skipped. A g_cost is subsequently established derived from whether, in the grid array, it is denoted as being near an obstacle ("3") in which case a steep penalty is awarded of 1000 as well as its inherent movement cost, which in this case is 1 due to the grid nature of the use case. If the neighbour cell is not previously explored, it is added to the unexplored list. If it has been explored but has a higher g_cost than the one stored in the array it is skipped entirely. Otherwise, the neighbour cell's parent arrays are updated with the current cells x/y positions as well as updating the "g_costs" array for the neighbour cell with its g_cost value. The "f_costs" array is also updated for the neighbour cell, providing a total estimated cost of navigating to the cell which includes its current g_cost as well as the value of its heuristic (using "getHeuristic"). This is iterated upon for all neighbour cells of all cells until the current cell is the goal cell.

The A* algorithm path is then obtained and returned by reversely searching the "parent_x" and "parent_y" arrays until arriving at the starting point.

CORE LOOP AND GRABBER FUNCTIONALITY

The controller implements the functionality of the A* algorithm and grid mapping in a core loop that runs successively, in a timing step of 64ms per loop. Sensor information is obtained from the distance and inertial sensors. A check if the robot is within 0.4m of the grabber is performed, halting the robot and signalling through the robots emitter that it is in position. Otherwise, the grid is mapped with the current values of the distance sensors. The A* algorithm is then performed to return the predicted path the robot should follow to reach the goal, with a check to see whether the path is empty which would cause the robot to cease movement. The Display is updated with the current state of the grid. Then, the path is converted from grid based coordinates to real simulation coordinates using the "gridToRealCoordinates" function which reverses the grid conversion (e.g. "real_x = (grid_x - 1) * cell_size - floor_x / 2 + cell_size / 2;") previously mentioned. The path however starts at the position of the robotic vehicle, which can cause for buggy movement and therefore a position 5 steps ahead in the path is chosen as the target position. The robotic vehicle then orients itself towards the target position using the "turnToTarget" function which returns a string direction which the robot must turn towards to face the target position, or if it is aligned utilising the inertial sensors given value for the robotic vehicles current yaw value. Finally, if aligned, the robot moves forward for a simulation step.

This functionality allows the robot to navigate its way around obstacles towards a grabber robot which was designed for the ACW1 (Weerasekara, 2026). The functionality of the grabber is largely unchanged except for the inclusion of emitter and receiver nodes which relay the position of the grabber to the robotic vehicle at the start of the simulation and awaits the signal from the robotic vehicle that it is in position. During which, the grabber picks up a box and places it on top of the robotic vehicle using its supervisor role to ascertain the GPS value of the vehicle.

TESTING AND CONCLUSION

The robotic system was tested in three custom made worlds with obstacles in each. A snapshot of each environment, is illustrated in Figure 4 below:

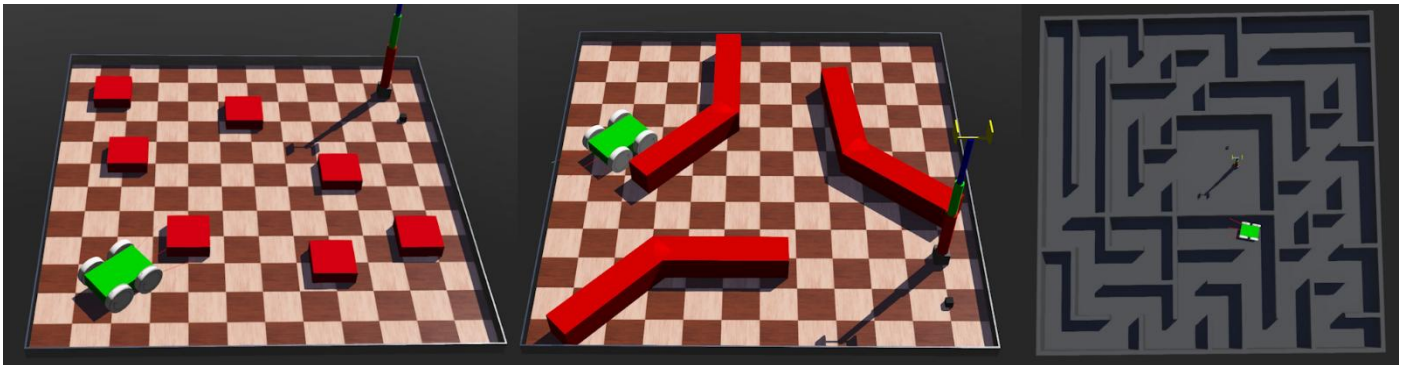


Figure 3 - Tested Environments for Robotic Vehicle

Test 1 features 7 uniform boxes placed in different positions. The robot was able to successfully navigate its way around the boxes to reach the grabber. Its movement was largely efficient with occasional deviation from the perfect path, and it reached the grabber in 38 second. Test 2 features 3 identical and large objects placed in positions intended to restrict the pathway of the robot to some degree. The robot was able to effectively manoeuvre its way to the grabber in 45 seconds. Task 3 is a more complicated test. It features a maze created using an online maze generator (Alance AB, 2026), which was converted to an OBJ 3D model and subsequently loaded in Webots as a Mesh shape node which allowed for extensive testing of the navigational system of the robot. The robot, with recorded deviations such as entering the wrong pathway, was eventually able to reach the grabber system in 8 minutes and 54 seconds. The robots movement was not erratic and clearly showed how the robot utilised the heuristic function of the A* algorithm to choose an optimal path when the layout of the environment was largely unknown. In all tests, the robotic vehicle was able to successfully communicate with the grabber (Weerasekara, 2026) and the target box was placed on the robotic vehicle.

In conclusion, the robotic vehicles operation was successful. It was able to navigate simple and complex environments to reach the end goal. Navigation was largely efficient with suitable obstacle avoidance. Implementation of the A* algorithm allowed the robotic vehicle to utilise the heuristic function successfully and find an optimal path when the vehicle did not have information about the environment it was navigating in. Further work might include utilising a lidar node instead of three distance sensors to obtain a larger spread of obstacle detection. Also, work should be implemented to maximise the efficiency of the system to reach the goal as fast as possible.

REFERENCES

Alance AB, 2026. *Maze Generator*. [Online]

Available at: <https://alance.se/>

[Accessed 10 05 2026].

Cyberbotics, 2026. *Simulating your robots with Webots*. [Online]

Available at: <https://cyberbotics.com/>

[Accessed 10 05 2026].

Hart, P. E., Nilsson, N. J. & Raphael, B., 1968. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), pp. 100-107.

Weerasekara, D., 2026. *MODELLING, ANALYSIS AND TRAJECTORY PLANNING OF A ROBOTIC MANIPULATOR*, Hull: The University of Hull.